BALANCING A BALL ON BEAM

FUTHER DEVELOPMENTS

CHEKKATTU NANDAN NIKHIL

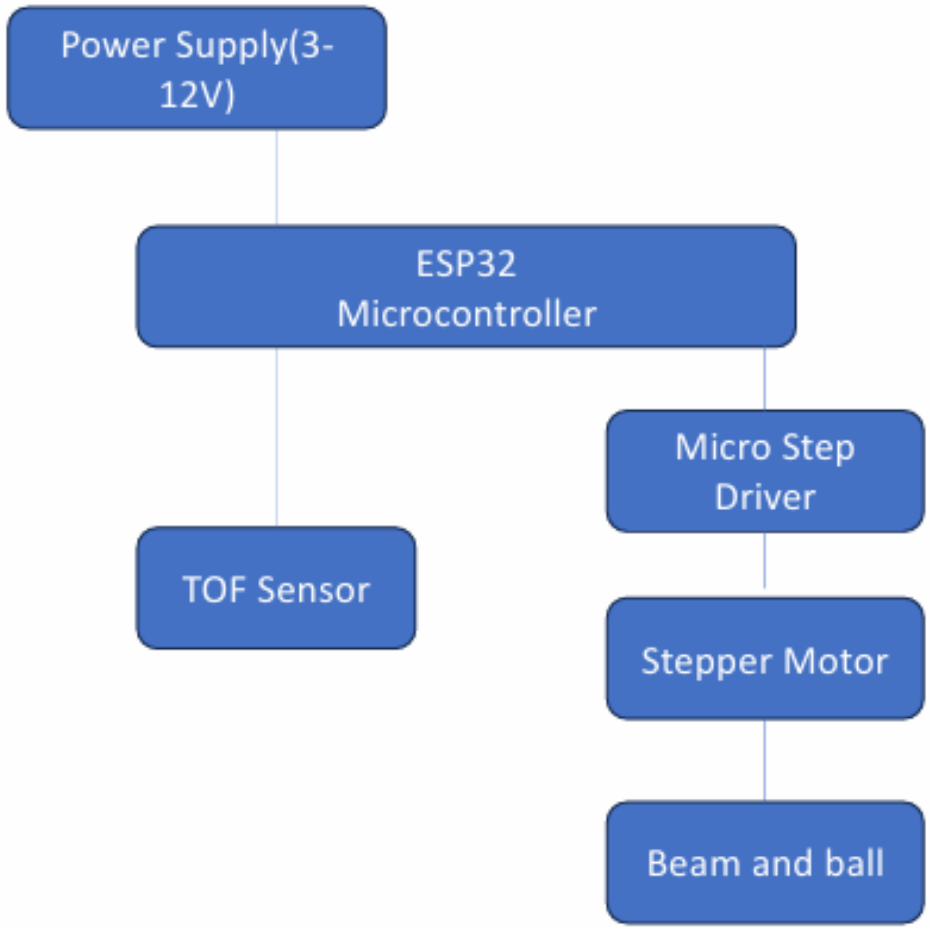
SURESH AKHILESH

MADHU AMAL

KULIRANIYIL SAJU JESTIN

System Architecture

Block Diagram



Step-by step Connection

1. Connecting the Stepper Motor to the Microstep Driver
- A+ and A- on the microstep driver to one coil of the motor.
- B+ and B- on the microstep driver to the other coil of the motor.

2. Powering the Microstep Driver
- VCC on the driver to the positive (+) terminal of the power supply.
- GND on the driver to the negative (-) terminal of the power supply.

3. Connecting the Microstep Driver to the ESP32
- PUL+ (Pulse): Sends pulses to make the motor move.
- DIR+ (Direction): Sets the direction (left or right) of motor rotation.
- ENA+ (Enable, optional): Enables or disables the motor.

Connections to ESP32:
- Connect PUL+ on the driver to any GPIO pin of the ESP32.
- Connect DIR+ on the driver to another GPIO pin of the ESP32.
- Connect PUL-, DIR-, and ENA- to the GND of the ESP32.

Step-by-step connection

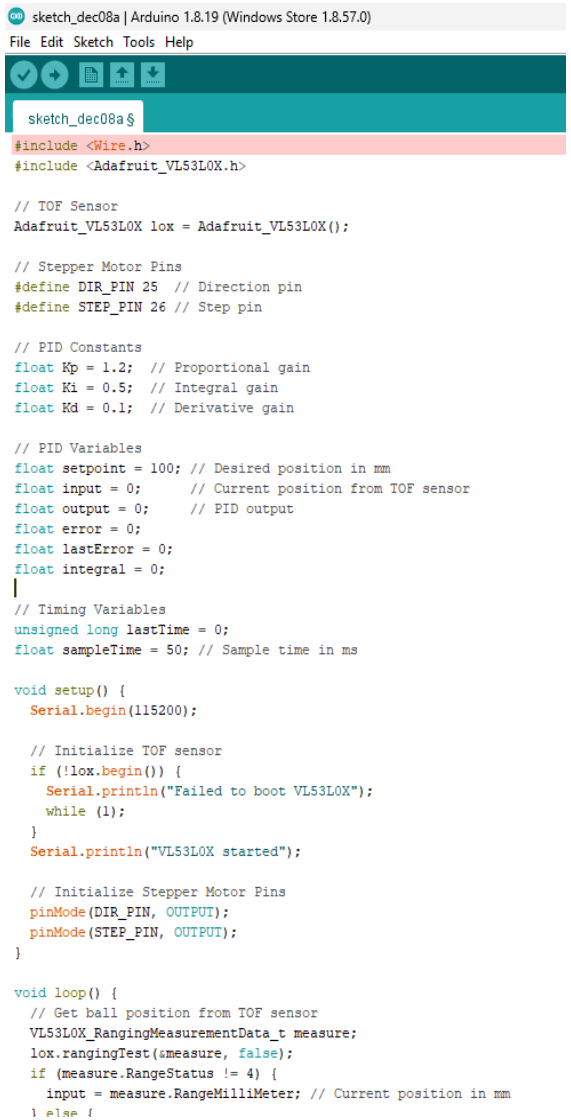
4. Connecting the TOF Sensor to the ESP32
Connect the TOF sensor’s VIN to the 3.3V pin on the ESP32.
Connect the GND of the TOF sensor to the ESP32 GND.

GND
SCL (Clock) pin on the TOF sensor to GPIO22 on the ESP32.
SDA (Data) pin on the TOF sensor to GPIO21 on the ESP32.

5. Powering the ESP32
Power the ESP32 using a USB cable connected to your computer or an external power source.

Final:
Common Ground: Make sure all components share the same ground (connect all GND pins together).
Motor Wiring: Ensure the motor wires are paired correctly.
Power Ratings: Confirm the motor driver and motor are rated for your power supply voltage.

For PID tuning




```ino
#include <Wire.h>
#include <Adafruit_VL53L0X.h>

// TOF Sensor
Adafruit_VL53L0X lox = Adafruit_VL53L0X();

// Stepper Motor Pins
#define DIR_PIN 25  // Direction pin
#define STEP_PIN 26 // Step pin

// PID Constants
float Kp = 1.2;  // Proportional gain
float Ki = 0.5;  // Integral gain
float Kd = 0.1;  // Derivative gain

// PID Variables
float setpoint = 100; // Desired position in mm
float input = 0;      // Current position from TOF sensor
float output = 0;     // PID output
float error = 0;
float lastError = 0;
float integral = 0;

// Timing Variables
unsigned long lastTime = 0;
float sampleTime = 50; // Sample time in ms

void setup() {
  Serial.begin(115200);

  // Initialize TOF sensor
  if (!lox.begin()) {
    Serial.println("Failed to boot VL53L0X");
    while (1);
  }
  Serial.println("VL53L0X started");

  // Initialize Stepper Motor Pins
  pinMode(DIR_PIN, OUTPUT);
  pinMode(STEP_PIN, OUTPUT);
}

void loop() {
  // Get ball position from TOF sensor
  VL53L0X_RangingMeasurementData_t measure;
  lox.rangingTest(&measure, false);
  if (measure.RangeStatus != 4) {
    input = measure.RangeMilliMeter; // Current position in mm
  } else {
    Serial.println("Out of range");
    return;
  }

  // PID Calculation
  unsigned long currentTime = millis();
  if (currentTime - lastTime >= sampleTime) {
    error = setpoint - input;
    integral += error * (currentTime - lastTime);
    float derivative = (error - lastError) / (currentTime - lastTime);

    output = Kp * error + Ki * integral + Kd * derivative;

    lastError = error;
    lastTime = currentTime;

    // Map output to motor control
    controlMotor(output);
  }

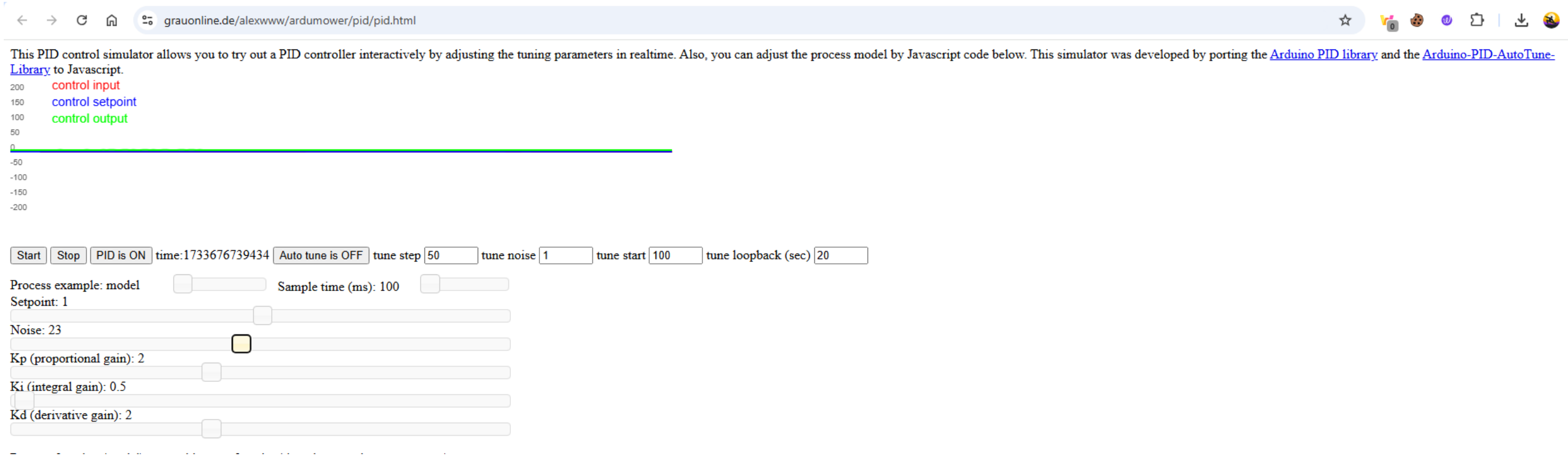
  // Debugging
  Serial.print("Input: ");
  Serial.print(input);
  Serial.print(" Output: ");
  Serial.println(output);
}

void controlMotor(float pidOutput) {
  // Convert PID output to stepper motor steps and direction
  int steps = abs(pidOutput);
  bool direction = pidOutput > 0;

  digitalWrite(DIR_PIN, direction); // Set direction
  for (int i = 0; i < steps; i++) {
    digitalWrite(STEP_PIN, HIGH);
    delayMicroseconds(500); // Adjust for motor speed
    digitalWrite(STEP_PIN, LOW);
    delayMicroseconds(500);
  }
}
```

For PID tuning

PID control Simulator



Thankyou